GDD · MicroJuego – Asteroides

Autor: Daniel Valero (individual)
Motor: Unity (proyecto 3D con vista ortográfica 2D)
Demo: carpeta `Demo/` → ejecutable de Windows (`.exe`). Descargar la carpeta completa: el `.exe` necesita la carpeta `_Data` y los `.dll` que lo acompañan.

1. Concepto

Shooter arcade inspirado en *Asteroids*. El jugador controla una nave que debe sobrevivir a meteoritos que entran desde los bordes de la pantalla. Al disparar a un meteorito, este se fragmenta en trozos más pequeños. Cada cierto tiempo llega una **oleada** con muchos meteoritos a la vez, cada vez más grande. El objetivo es conseguir la máxima puntuación antes de perder las 3 vidas.

Controles

Acción	Tecla
Avanzar / retroceder	W / S (o flechas ↑ ↓)
Girar	A / D (o flechas ← →)
Estabilizar (frenado suave)	Shift
Disparar	Espacio
Pausa / reanudar	Esc o P
Reiniciar (en pausa o Game Over)	R

2. Escenas

El juego tiene una única escena, **`Game`** (`Assets/Scenes/Game.unity`), que contiene todo el bucle de juego. Las "pantallas" de pausa y Game Over no son escenas separadas, sino paneles de interfaz que se activan y desactivan sobre la propia escena. El reinicio recarga la escena `Game`.

Cámara: el proyecto es 3D, pero la `Main Camera` usa proyección **ortográfica**, posición (0, 0, -10) y mira al plano X-Y. Todo el juego ocurre en Z = 0, por lo que se ve como un juego 2D pero usando la física 3D de Unity.

3. Assets y GameObjects

Assets

Carpeta	Contenido
`Assets/Sprites`	Sprites proporcionados (nave, bala, meteorito, fondo). Importados como *Sprite (2D and UI)* con *Pixels Per Unit* ajustado por sprite para escalar cada elemento.
`Assets/Prefabs`	`Bullet`, `Enemy`, `Explosion`
`Assets/Scripts`	Código C# del juego (ver sección 5)
`Assets/Scenes`	Escena `Game`

GameObjects de la escena

GameObject	Componentes principales	Función
`Main Camera`	Camera (Orthographic)	Vista 2D del juego
`Background`	SpriteRenderer (Order in Layer -10), `BackgroundDrift`	Fondo con leve movimiento de ambientación

GameObject	Componentes principales	Función
`Player`	SpriteRenderer, Rigidbody, SphereCollider, `PlayerMovement`, `PlayerShooting`, `PlayerHealth`	Nave del jugador
`EnemySpawner`	`EnemySpawner`	Genera meteoritos y oleadas
`GameManager`	`ScoreManager`, `GameController`, `EnemyPool`	Puntuación, estados del juego y pool de enemigos
`Canvas`	Canvas Scaler (Scale With Screen Size)	Interfaz: `ScoreText`, `LivesText`, `PausePanel`, `GameOverPanel`

Prefabs

Prefab	Componentes	Notas
`Bullet`	SpriteRenderer, Rigidbody (sin gravedad, Z bloqueado), SphereCollider (*Is Trigger*), `Bullet`	Atraviesa objetos y detecta impactos por trigger
`Enemy`	SpriteRenderer, Rigidbody (sin gravedad, Z bloqueado), SphereCollider, `Enemy` · Tag `Enemy`	Collider algo más pequeño que el sprite para que los choques sean justos
`Explosion`	Particle System (burst de 30 partículas, sin loop, *Stop Action: Destroy*)	Efecto al destruir meteoritos

Configuración física común: todos los Rigidbody tienen *Use Gravity* desactivado y *Freeze Position Z*, para que la física 3D nunca saque objetos del plano de juego. La nave además bloquea todas las rotaciones físicas (su giro se controla por código), evitando que los choques la hagan girar.

4. Mecánicas

Obligatorias

- **Movimiento:** la nave se mueve con fuerzas físicas (`AddForce`) en la dirección a la que apunta y gira sobre el eje Z. Tiene inercia controlada mediante *Linear Damping* y una velocidad máxima (`ClampMagnitude`). No puede salir de los límites de la pantalla, que se calculan a partir del tamaño de la cámara ortográfica.
- **Estabilización:** manteniendo Shift, la velocidad se reduce progresivamente hacia cero (`Vector3.Lerp`), para un frenado suave en lugar de en seco.
- **Disparo:** cada pulsación de Espacio crea una bala delante del morro de la nave, con su misma rotación. La bala avanza en línea recta y desaparece al salir de pantalla o al impactar.
- **Spawn de enemigos:** los meteoritos aparecen fuera de pantalla, en un borde aleatorio, y se dirigen hacia una zona aleatoria cercana al centro para cruzar el área de juego. Tienen velocidad y giro aleatorios.
- **Puntuación:** cada meteorito destruido suma puntos, mostrados en pantalla.

Extras

- **Fragmentación:** cada meteorito tiene un tamaño (3 = grande, 2 = mediano, 1 = pequeño). Al recibir un disparo, si no es del tamaño mínimo, se divide en 2 fragmentos más pequeños que salen en direcciones aleatorias.
- **Oleadas:** además del goteo constante (un meteorito cada 2 s), cada 15 s llega una oleada con muchos meteoritos casi a la vez. Cada oleada es mayor que la anterior.
- **Vidas:** la nave tiene 3 vidas. Al chocar con un meteorito pierde una y queda invulnerable 1,5 s, parpadeando. Con 0 vidas, Game Over.
- **Pausa / Reinicio / Game Over:** pausa con Esc o P (congela el tiempo con `Time.timeScale = 0`), reinicio con R recargando la escena.
- **Efectos:** explosión de partículas al destruir meteoritos y fondo con movimiento suave.

5. Scripts

Script	Objeto	Responsabilidad
`PlayerMovement`	Player	Lectura de teclado (Input System), aceleración, giro, velocidad máxima, estabilización con Shift y límites de pantalla
`PlayerShooting`	Player	Disparo con Espacio (bloqueado en pausa)
`PlayerHealth`	Player	Vidas, detección de choque con meteoritos (`OnCollisionEnter`), invulnerabilidad con parpadeo (corrutina) y muerte
`Bullet`	Prefab Bullet	Movimiento de la bala, destrucción fuera de pantalla e impacto con enemigos (`OnTriggerEnter`)
`Enemy`	Prefab Enemy	Lanzamiento (`Launch`), recepción de impacto (`Hit`), fragmentación, explosión y retorno al pool
`EnemySpawner`	EnemySpawner	Generación continua y por oleadas crecientes (corrutina)
`EnemyPool`	GameManager	Object Pool de meteoritos
`ScoreManager`	GameManager	Puntuación y texto de UI
`GameController`	GameManager	Estados de juego: pausa, Game Over y reinicio
`BackgroundDrift`	Background	Balanceo suave del fondo con seno/coseno

Patrones de diseño

Object Pooling (`EnemyPool`). Crear y destruir objetos (`Instantiate`/`Destroy`) reserva y libera memoria; hacerlo con muchos objetos a la vez, como en una oleada, provoca tirones. Por eso, al cargar la escena se crean 60 meteoritos desactivados. Cuando el spawner o una fragmentación necesita uno, `Get()` activa uno libre y lo reinicia (posición, rotación, escala y tamaño). Al morir o salir de pantalla, `Return()` lo desactiva para reutilizarlo. Si en algún momento todos están en uso, el pool crea uno más y lo añade, sin límite fijo. Así las oleadas grandes no generan picos de rendimiento.

Singleton. `ScoreManager`, `GameController` y `EnemyPool` exponen una referencia estática `Instance`, de modo que cualquier script puede acceder a ellos (por ejemplo, `ScoreManager.Instance.AddPoints(10)`) sin tener que buscarlos en la escena.

Decisiones técnicas

- **`Awake` en lugar de `Start` en `Enemy`:** el spawner llama a `Launch()` inmediatamente después de obtener el enemigo, por lo que el Rigidbody debe estar ya referenciado.
- **`Hit()` en el enemigo, no en la bala:** la bala solo notifica el impacto; el enemigo decide qué ocurre (puntos, fragmentación, explosión). Esto permitió añadir la fragmentación sin tocar la bala.
- **Límites de pantalla calculados por posición** en lugar de `OnBecameInvisible`, que en el editor también depende de la vista *Scene*.
- **Input System nuevo** (`Keyboard.current`), ya que es el sistema activo por defecto en el proyecto.

6. Interacción

Elemento A	Elemento B	Tipo	Resultado
Bala	Meteorito	Trigger	El meteorito suma puntos, se fragmenta (si su tamaño es mayor que 1) y explota. La bala desaparece.
Nave	Meteorito	Colisión física	El meteorito explota. La nave pierde una vida (si no es invulnerable) y parpadea.
Nave	Borde de pantalla	Por código	La nave se detiene en el borde.
Bala / Meteorito	Fuera de pantalla	Por código	La bala se desactiva y el meteorito vuelve al pool.
Jugador	Interfaz	Teclado	Esc/P pausa, R reinicia. El marcador y las vidas se actualizan en tiempo real.

Flujo de juego: Inicio → partida (goteo de meteoritos + oleadas crecientes) → pausa opcional (Esc/P) → Game Over al perder las 3 vidas → reinicio (R).